Détection de fraude financière par Machine Learning

Règles métier contre intelligence artificielle : laquelle détecte le mieux la fraude, et à quel coût pour une banque ?

Projet de fin d'études (PFE) – Master 1 Finance, HESTIM – année universitaire 2025–2026. Auteure : **Imane Aghanim** · Encadrante : **Mme Fatima Berrada**

English summary: comparison of a hand-written business rule, a Random Forest and a tuned XGBoost for fraud detection on the simulated PaySim dataset, with results translated into a (hypothetical) cost in MAD.

Question de recherche

> L'IA détecte-t-elle mieux la fraude que les règles métier, et à quel coût pour une banque marocaine ?

Trois sous-questions :

1. Les règles métier suffisent-elles ?
2. L'IA est-elle fiable, et quelle méthode détecte le mieux ?
3. Quel est le meilleur compromis entre qualité de détection et coût ?

Résultats principaux

Jeu de test : 314 573 transactions, dont 343 fraudes.

Méthode	Fraudes trouvées	Fraudes manquées	Fausses alertes	Précision	Rappel	F1	Coût (MAD fictifs)
Règle métier (≥ 4 critères sur 6)	232	111	18 914	1,2 %	67,6 %	0,024	8 387 100
Random Forest	273	70	12	95,8 %	79,6 %	0,869	3 501 800
XGBoost réglé	**295**	**48**	**28**	**91,3 %**	**86,0 %**	**0,886**	**2 404 200**

- Les deux modèles d'IA battent nettement la règle (F1 de 0,87 à 0,89 contre 0,02) et coûtent plus de 2 fois moins cher (2,4 fois pour Random Forest, 3,5 fois pour XGBoost réglé).
- XGBoost réglé est environ 31 % moins cher que le Random Forest, pour un F1 proche.
- La règle trouve 68 % des fraudes mais déclenche 18 914 fausses alertes : inutilisable en pratique.

Robustesse

- Validation croisée à 5 parts (score F1) : Random Forest 0,84 à 0,89 ; XGBoost par défaut 0,40 à 0,87 (instable) ; XGBoost réglé 0,87 à 0,91.
- Sur 5 autres découpages, XGBoost réglé est moins cher 5 fois sur 5.
- Test de McNemar sur les fraudes : 22 fraudes trouvées en plus, aucune en moins ($p < 0,001$).

Données

- **PaySim** : jeu de transactions de paiement mobile **simulées** (disponible publiquement, par exemple sur Kaggle).
- Échantillon utilisé : 1 048 575 transactions, dont 1 142 fraudes (0,11 %).
- Les données ne sont pas incluses dans ce dépôt : télécharger PaySim séparément.
- Nettoyage : suppression des numéros de compte, type d'opération encodé en chiffre, aucune valeur manquante.
- Découpage stratifié : 70 % entraînement, 30 % test (`random_state=42`).

Méthodes comparées

Méthode	Principe	Paramètres
Règle métier	Alerte si au moins 4 critères sur 6 sont réunis : montant > 200 000, compte vidé, type TRANSFER ou CASH_OUT, destinataire à solde 0, solde destinataire non crédité, opération de nuit	Seuil : 4 critères
Random Forest	Bagging : 100 arbres indépendants, vote à la majorité	`n_estimators=100`, `class_weight='balanced'`
XGBoost réglé	Boosting : arbres construits un par un, chacun corrige les erreurs du précédent	`n_estimators=100`, `max_depth=3`, `scale_pos_weight=10`, `tree_method='hist'`

Seuil de décision des modèles : 0,5.

Mesures

- **Précision** = VP ÷ (VP + FP) · **Rappel** = VP ÷ (VP + FN) · **F1** = 2 × P × R ÷ (P + R)
- **Coût** = fraudes manquées × 50 000 + fausses alertes × 150 (MAD)
    - Exemple, XGBoost réglé : 48 × 50 000 + 28 × 150 = 2 404 200 MAD.

Le pourcentage de bonnes réponses n'est pas utilisé : répondre toujours « pas de fraude » donnerait 99,9 % et serait inutile.

Limites

- Données **simulées** (PaySim) : la fraude y est très structurée par type d'opération, les signaux sont probablement plus nets que dans une vraie banque.
- Les coûts (50 000 MAD par fraude manquée, 150 MAD par fausse alerte) sont des **hypothèses**, sans coût de mise en place. Les montants exacts sont fictifs ; ce qui compte est la comparaison entre méthodes.
- La règle métier a été construite pour l'étude, ce n'est pas une règle bancaire réelle.
- Random Forest non réglé ; XGBoost réglé sur 8 combinaisons seulement.
- Seuil de décision laissé à 0,5 : un seuil plus bas réduit le coût mais fait chuter le F1.

Perspectives

- Projet pilote avec de vraies données et de vrais coûts.
- Expliquer les décisions du modèle (valeurs de Shapley).
- Optimiser le seuil de décision.
- Respecter le cadre réglementaire (loi 09-08 sur les données personnelles).

Outils

Python, pandas, scikit-learn, XGBoost, Jupyter.

Contenu du dépôt

Fichier	Rôle
`PFE_fraude_notebook.ipynb`	Notebook complet, avec les résultats affichés : comparaison, robustesse, analyses complémentaires
`fraude_paysim.py`	Même démarche en script : comparaison finale + tests de robustesse
`requirements.txt`	Bibliothèques Python nécessaires
`README.md`	Ce fichier

Reproduire les résultats

1. Télécharger PaySim et noter le chemin du fichier CSV.
2. Installer les bibliothèques : `pip install -r requirements.txt`
3. Lancer le script : `python fraude_paysim.py chemin/vers/paysim.csv` (environ 6 minutes ; ajouter `--rapide` pour sauter les tests de robustesse, environ 1 minute 30).
4. Ou ouvrir le notebook, changer `CHEMIN` dans la première cellule de code, puis tout exécuter (environ 10 minutes).

Le script écrit le tableau comparatif dans `resultats.csv`. Les résultats doivent être identiques à ceux de ce README (graine fixe : `random_state=42`).

Licence

Code publié sous licence MIT (voir le fichier `LICENSE`).